

CV Sorting using LLMs

Capstone Project CS[02]

Abstract

Recruiters spend substantial time reading unstructured resumes against loosely written job descriptions. This capstone implements a terminal-based CV sorting system that ranks candidate documents for a role using two complementary large language models. The first model (LLM-1) extracts a structured job profile and structured candidate profiles from PDF, DOCX, Markdown, or plain-text files. The second model (LLM-2) scores each candidate with an explicit rubric covering skills, experience, domain fit, and education, and then recalibrates scores across the shortlist so independently judged CVs stay comparable. A deterministic, word-boundary keyword-overlap signal is injected into the ranker prompt as evidence, so obvious gaps cannot be ignored while the final judgement remains with the reasoning model. The program starts from `main.py`, takes the API key only at runtime, prints a ranked table with per-candidate explanations, and supports recruiter filters and JSON export. Unreadable CVs or failed model calls are reported and skipped instead of aborting the run. On a labelled test set of five CVs in four formats, including a keyword-stuffed decoy, the system reproduced the expected ordering exactly in three independent runs, with per-candidate score variation of at most six points.

1. Introduction

Applicant tracking still depends on keyword filters or manual reading. Both miss context: a candidate may describe relevant work without naming a tool, or list a skill with no evidence of using it. Instruction-tuned LLMs can parse messy documents and justify a match, but a single model asked to rank CVs mixes extraction errors with scoring bias. This project therefore separates extraction from judgement, which also gives each of the two required LLMs a clear role. The work is a command-line tool because evaluation is terminal-based and a GUI would not improve the ranking objective.

2. Problem Statement

Given one job description and N candidate CVs in common document formats, produce a stable ranked list with overall scores, missing must-have skills, and a written rationale per candidate, without embedded secrets or a front-end, runnable with the evaluator's own API key and test files.

3. Objectives

- Rank multiple CVs against one job description from the terminal.
- Use two LLMs with distinct roles: extraction (LLM-1) and scoring (LLM-2).
- Accept PDF, DOCX, Markdown and text; tolerate bad files without aborting.
- Return inspectable scores, missing skills, and optional JSON output.

4. Methodology and Implementation

The pipeline is linear and started by `main.py`. `document_loader.py` reads UTF-8 text and Markdown, PDFs via `pdfplumber`, and DOCX via ZIP/XML parsing (no python-docx dependency); hidden and unsupported files are ignored, and the JD is excluded if it sits in the CV folder. `llm_client.py` wraps the OpenAI chat-completions SDK with an optional base URL, so the same code runs against OpenAI, Gemini, Groq, or a local Ollama server. It requests JSON mode (falling back when unsupported), repairs malformed JSON once, retries transient errors, and fails immediately on authentication or unknown-model errors. LLM-1 fills a fixed JSON schema for the job and for each CV at temperature 0. Before scoring, `extractor.overlap_evidence` computes must-have and nice-to-have coverage. Matching is case-insensitive on word boundaries, tolerates vendor prefixes (Apache Spark vs Spark) and suffix compounds (PySpark vs Spark), and rejects false positives such as R in Spark or Java in JavaScript. LLM-2 receives this evidence with both JSON profiles and returns 0-100 sub-scores and an overall score weighted skills 40%, experience 30%, domain 20%, education 10%, plus strengths, risks, and an explanation. A calibration call then lets LLM-2 see the whole shortlist and adjust overall scores for consistency. Results are filtered (`--required-skills`, `--min-score`) and sorted by overall score, then skills score, then fewer missing must-haves. A failure on one CV is logged and skipped; the program exits with an error only if the JD cannot be processed or no CV could be scored.

5. Model Selection

Model choice follows the task split. Extraction is schema filling, where latency and cost dominate, so a small fast model is used: `gpt-4o-mini` by default, `gemini-3.5-flash-lite` in the validation runs below. Ranking needs trade-off reasoning and trustworthy explanations, so a stronger model is used: `gpt-4o` by default, `gemini-3.8-flash` in validation. The same small-extractor / strong-ranker split is available on Groq (Llama 3.1 8B and Llama 3.3 70B) or Ollama for fully local, privacy-preserving use.

6. Results and Analysis

The system was evaluated on a labelled set: one Senior Data Engineer JD and five CVs in PDF, DOCX, TXT and MD, with a human reference order. Sara Khan is a deliberate decoy who lists every JD keyword (100% keyword coverage) but whose history is entirely non-technical. Table 1 shows overall scores across two full runs and one run without calibration.

Table 1. Overall score (0-100) per candidate; all runs ranked in expected order.

Candidate (format)	Expected	Run 1	Run 2	No calibration
Priya Sharma (PDF)	1	96	97	97
Rahul Verma (DOCX)	2	71	72	70
Emily Chen (TXT)	3	39	43	37

Sara Khan, decoy (TXT)	4	25	22	20
John Doe (MD)	5	13	16	17

All three runs reproduced the reference order exactly (Kendall tau = 1.0), and no score moved by more than six points between runs. The decoy was ranked fourth with a score of 20-25: LLM-2 explicitly flagged her skills as self-reported with no supporting work, which shows the value of separating keyword evidence from judgement. Calibration changed scores by at most six points and did not change the order on this set, so its benefit is mainly for larger shortlists. With a corrupt DOCX and an empty file added, both were skipped with warnings and the other CVs were still ranked; the filters `--required-skills airflow` `--min-score 50` correctly left only the top candidate. An offline suite of 41 unit tests (fake LLMs) covers loaders, JSON repair, retries, matching, ranking, filters, and CLI errors. A run takes about 80-100 seconds for five CVs because calls are sequential.

7. Conclusion and Future Work

The system meets the capstone objective: CVs and a job description in, a ranked and explained list out, using two role-specialised LLMs. Future work: parallel model calls for large CV sets, embedding pre-filtering, OCR for scanned PDFs, and a recruiter feedback file that re-runs only the ranking stage.

References

1. OpenAI Python SDK, Chat Completions API.
2. Google Gemini OpenAI-compatible API.
3. Groq and Ollama OpenAI-compatible endpoints.
4. pdfplumber.
5. Capstone brief: CV Sorting using LLMs (CS[02]).